CoSc3091 - Web Programming Individual Assignment I Report

Name : Getachew tlahun

ID : 057/16

Project Title: Campus Lost and Found Portal

Course Code: CoSc3091

Submitted To: Department of Computer Science

1. Executive Summary & Overview

1.1 Project Overview

The **Campus Lost and Found Portal** is a full-stack, dynamic web application designed and developed to provide a centralized digital platform for university students, faculty, and administrative staff. Within a busy campus environment, students frequently misplace personal belongings such as ID cards, textbooks, electronics, and keys. Traditional lost-and-found methods (such as physical bulletin boards or inquiry desks) are often inefficient, location-restricted, and slow.

This project addresses those challenges by providing a 24/7 accessible web-based portal. Users can register an account, log in, browse through missing and found items, post new listings with contact information, and send direct inquiries to system administrators.

1.2 Learning Objectives Achieved

Through the development of this project, several key competencies defined in the CoSc3091 course guidelines were successfully applied:

- **Semantic HTML5:** Structuring clean, accessible, and multi-page layouts using semantic elements (<header>, <nav>, <main>, <section>, <footer>).
- **Responsive CSS3 Styling:** Employing CSS Variables, Flexbox, CSS Grid layouts, and media queries to deliver a modern visual UI across mobile, tablet, and desktop viewports.
- **Client-Side Interactivity (JavaScript):** Implementing client-side form validation and dynamic DOM manipulations to enhance user experience before sending payloads to the server.
- **Server-Side Logic (PHP):** Handling HTTP GET/POST requests, session management, user authentication, and data routing using modular PHP scripts.
- **Relational Database Management (MariaDB/MySQL):** Designing a relational database schema, handling foreign key relationships, and executing secure SQL queries using PHP Data Objects / Prepared Statements.
- **Version Control & Deployment:** Managing code increments via Git/GitHub commits and maintaining documentation.

2. System Architecture & Folder Structure

2.1 Technology Stack

- **Front-End:** HTML5, CSS3 (Custom Grid & Flexbox), JavaScript (ES6 Native DOM Validation)
- **Back-End:** PHP 8.x (Modular execution, Session management)
- **Database Engine:** MariaDB / MySQL (Using Prepared Statements for security)
- **Environment:** Apache / Local Web Server (XAMPP / MariaDB environment)

2.2 Project Folder Architecture

The project follows a clean separation of concerns, separating backend configuration, shared layout components, styles, scripts, and individual page handlers:

Plaintext

```
campus_lost_found/
├── config/
│   └── db.php          # Relational Database Connection Logic
├── css/
│   └── style.css       # Responsive Layout & Global Custom Theme
├── js/
│   └── main.js         # Client-Side Form Validation Scripts
├── includes/
│   ├── header.php      # Dynamic Navbar with Session Control
│   └── footer.php      # Global Page Footer Layout
├── index.php           # Page 1: Landing Page (Hero, Features)
├── about.php           # Page 2: Student Profile & System Info
├── items.php           # Page 3: Core Service Page (Browse Items)
├── report_item.php     # Page 4: Core Functionality (Post Item)
├── contact.php         # Page 5: Interactive Contact & Inquiry Form
├── register.php        # User Account Creation Page
├── login.php           # User Sign-In Page
├── logout.php          # Session Termination Logic
└── schema.sql          # Database Export Script (DDL & Sample Data)
```

3. Database Design & Entity Relationship (ER) Structure

The backend persistence layer consists of three main tables linked via foreign keys and constraints to maintain data integrity.

3.1 Relational Schema Definitions

Table 1: users

Stores student and staff credentials with encrypted authentication tokens.

- id (INT, Primary Key, Auto Increment)

- full_name (VARCHAR 100, NOT NULL)
- email (VARCHAR 100, Unique, NOT NULL)
- username (VARCHAR 50, Unique, NOT NULL)
- password (VARCHAR 255, Hashed using PASSWORD_BCRYPT)
- created_at (TIMESTAMP, Default Current Timestamp)

Table 2: items

Stores the lost and found listings posted by authenticated users.

- id (INT, Primary Key, Auto Increment)
- user_id (INT, Foreign Key referencing users(id) ON DELETE CASCADE)
- title (VARCHAR 150, NOT NULL)
- category (ENUM('Lost', 'Found'), NOT NULL)
- description (TEXT, NOT NULL)
- location (VARCHAR 150, NOT NULL)
- contact_phone (VARCHAR 20, NOT NULL)
- date_reported (TIMESTAMP, Default Current Timestamp)

Table 3: messages

Stores contact inquiries submitted through the contact page.

- id (INT, Primary Key, Auto Increment)
- sender_name (VARCHAR 100, NOT NULL)
- email (VARCHAR 100, NOT NULL)
- message (TEXT, NOT NULL)
- sent_at (TIMESTAMP, Default Current Timestamp)

3.2 SQL DDL Code (schema.sql)

SQL

```
CREATE DATABASE IF NOT EXISTS campus_lost_found;
USE campus_lost_found;

CREATE TABLE IF NOT EXISTS users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE,
    username VARCHAR(50) NOT NULL UNIQUE,
    password VARCHAR(255) NOT NULL,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS items (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
```

```
    title VARCHAR(150) NOT NULL,
    category ENUM('Lost', 'Found') NOT NULL,
    description TEXT NOT NULL,
    location VARCHAR(150) NOT NULL,
    contact_phone VARCHAR(20) NOT NULL,
    date_reported TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

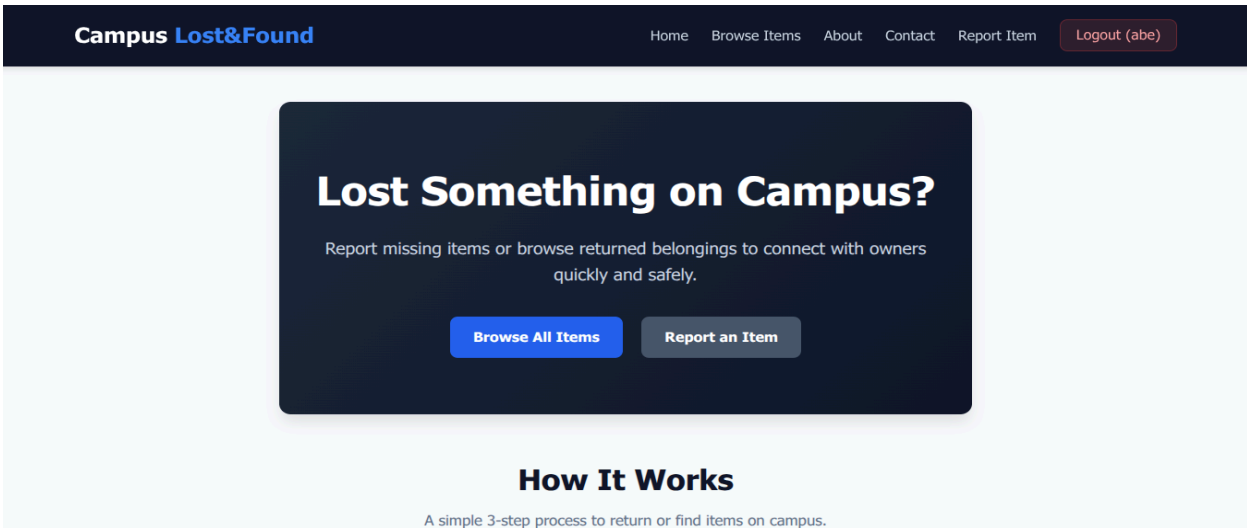
CREATE TABLE IF NOT EXISTS messages (
    id INT AUTO_INCREMENT PRIMARY KEY,
    sender_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    message TEXT NOT NULL,
    sent_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
```

4. Key System Features & Functional Modules

4.1 Page Implementations (5+ Linked Pages Requirements)

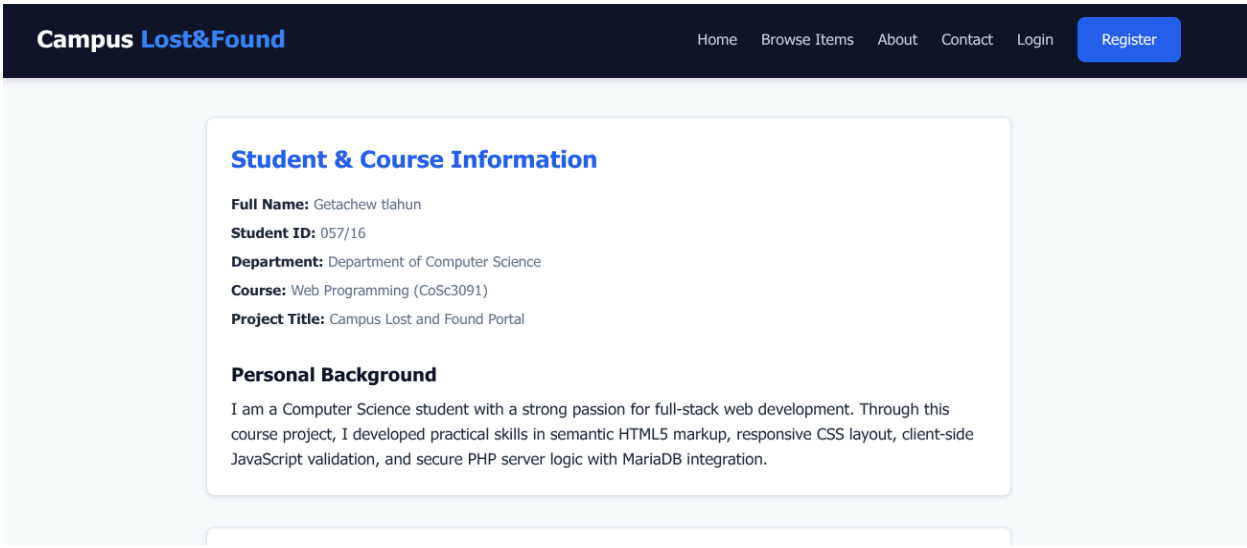
The web application fulfills and exceeds the minimum page requirements specified in Section 3.1 of the assignment rubric:

1. **Home Landing Page (index.php):**
    - Features a hero banner introducing the platform's primary purpose.
    - Includes dynamic call-to-action buttons redirecting users to browse or report items.
    - Contains a 3-step feature breakdown grid explaining the usage workflow.

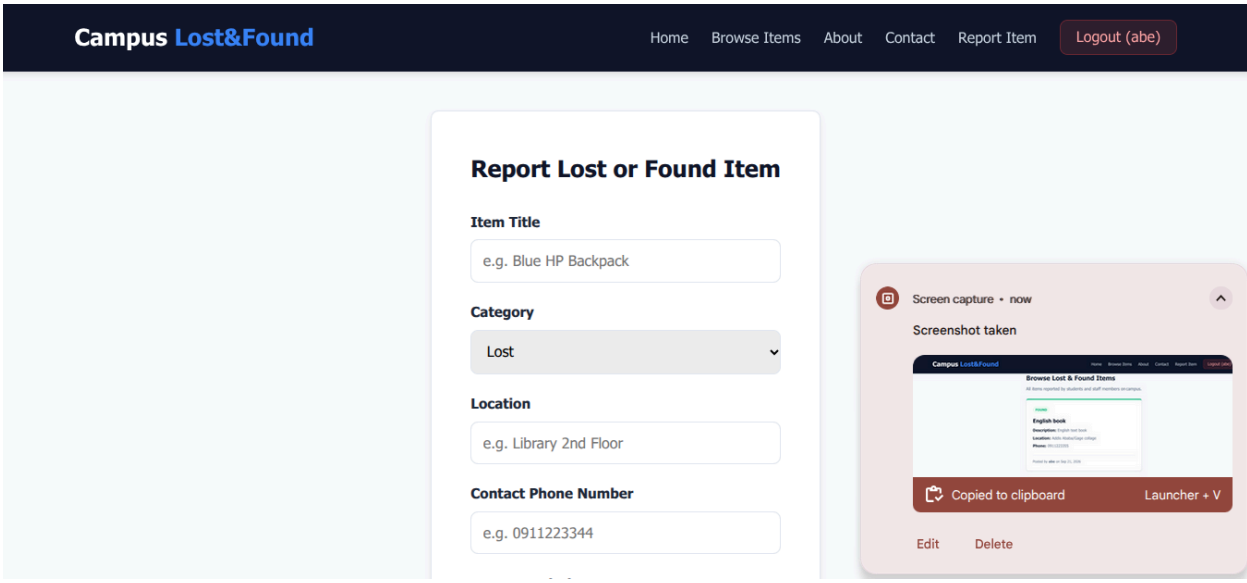


2. **About Page (about.php):**

- Fully satisfies the course criteria by presenting the developer's academic identity.
- Includes student name, ID number, department (Computer Science), and course code (CoSc3091).
- Displays a personal statement explaining background, technical motivations, and reason for selecting this project.

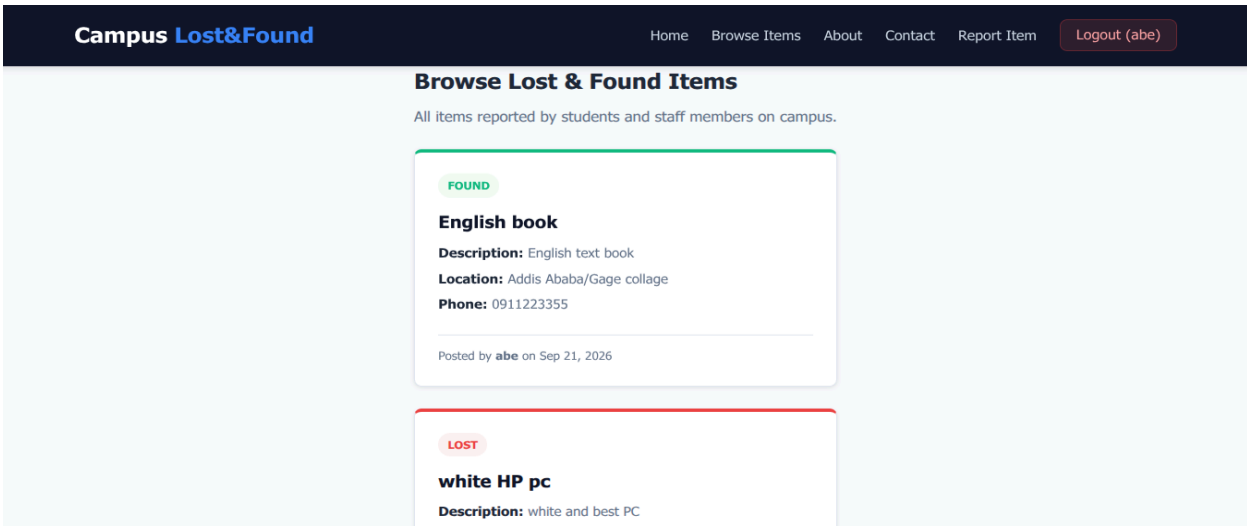


3. **Browse Items / Core Listing Page (items.php):**
   - Serves as the primary core listing directory.
   - Queries the items table via an inner join with users to display listing cards.
   - Uses distinctive visual badges and color borders to separate **Lost** items (Red border) from **Found** items (Green border).

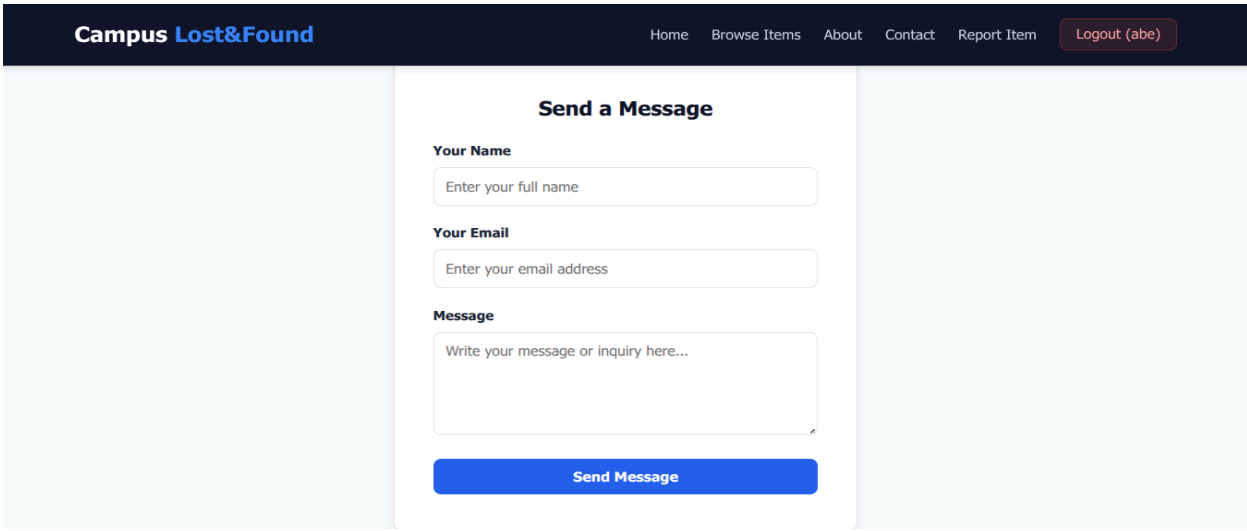


4. **Report Item Page (report_item.php - Core Functionality):**

- Acts as the primary interactive write functionality required by Section 3.3.
- Protected by session authentication: unauthenticated guests are automatically redirected to login.php.
- Form submits title, category, location, phone, and detailed descriptions directly into MariaDB via prepared SQL queries.



5. **Contact Page (contact.php):**
   - Provides campus office locations, support email addresses, phone numbers, and operational hours.
   - Contains an active contact form that persists user feedback into the messages table.



6. **Authentication Pages (register.php, login.php, logout.php):**
   - **Registration:** Validates inputs, checks for duplicate accounts, and securely hashes passwords with password_hash().

    - **Login:** Verifies stored hash signatures using password_verify() and initializes session identifiers ($_SESSION['user_id']).
    - **Conditional Header Navigation:** Automatically toggles navigation buttons—showing **Login / Register** for guest users, and **Report Item / Logout (username)** for authenticated users.

5. Security Protocols & Validation Mechanisms

To ensure system integrity and score high on security criteria, the following measures were embedded:

5.1 Defense Against SQL Injection

All user inputs submitted via forms are processed using PHP Prepared Statements ($conn->prepare()) and bound parameters ($stmt->bind_param()). Raw SQL concatenation is strictly avoided.

5.2 Password Protection Strategy

Plaintext passwords are never stored in the database. The system employs the standard BCrypt algorithm (PASSWORD_BCRYPT), ensuring cryptographic resistance against rainbow table and brute-force attacks.

5.3 Cross-Site Scripting (XSS) Mitigation

Data retrieved from the database is escaped using htmlspecialchars() before rendering onto the DOM, preventing unauthorized HTML or JavaScript injection scripts.

5.4 Dual-Layer Input Validation

- **Client-Side (JavaScript):** Intercepts form submit events, trims whitespace, and highlights empty required fields dynamically before sending requests.
- **Server-Side (PHP):** Re-evaluates email formats using filter_var(), checks string lengths, and ensures mandatory fields are populated prior to database execution.